

PROJECT PROPOSAL

RideShield

Smart Helmet with IoT for Motorcycle Safety

IoTrix 2.0 Semi-Final Competition - Track A: Embedded IoT System

Team	RideShield
Team Leader	Piragenth
Institution	South Eastern University of Sri Lanka - Faculty of Technology - Department of ICT
Competition	IoTrix 2.0 Semi-Final
Track	Track A - Embedded IoT System
Date	5 October 2026

Design principle: The safety-critical crash and emergency path remains independent of camera/AI processing. Vision failures must not block crash detection, local logging or emergency communication.

Problem + Proof + Understanding

1. Executive Summary

RideShield is a distributed embedded-IoT smart helmet platform intended to improve motorcycle rider safety before, during and after a hazardous event. The current design combines a deterministic safety controller with separate computer-vision nodes. The safety controller verifies helmet use, monitors motion using an MPU6050, classifies possible crash events, provides a rider-cancel countdown, records black-box data and sends an emergency alert with location through cellular communication. The front camera subsystem detects potholes and estimates visible road wetness; the rear camera subsystem is intended to provide rear-vehicle awareness. Secondary rider-monitoring functions include prolonged eye-closure detection and pre-ride alcohol-vapour screening.

2. Problem Definition

- After a motorcycle crash, emergency response can be delayed because the event, location and severity are not immediately communicated.
- Potholes and visibly wet road surfaces can increase riding risk, especially when combined with higher speed or poor visibility.
- Rear vehicles can be difficult to monitor continuously while riding.
- A helmet may be worn incorrectly or not properly fastened, reducing the value of the safety system.
- Safety features must continue operating even if non-critical AI/camera functions fail.

3. Objectives and Semi-Final Scope

Priority	Objective	Semi-final target
Core	Detect and validate possible crash events	Real embedded proof using MPU6050 and state-machine logic
Core	Allow false alarms to be cancelled	Physical cancel button + countdown
Core	Record and communicate an emergency event	GPS/LTE/microSD integration as available
Core	Verify that the helmet is actually worn	Two-signal helmet presence + strap logic
POC	Detect potholes ahead	Camera-based software proof-of-concept
POC	Estimate visible road wetness	Dry/damp/wet/water classifier proof-of-concept
Staged	Rear vehicle awareness	Architecture and initial camera work if available
Staged	Eye closure and alcohol screening	Secondary integration after the core path is stable

Scope control: The semi-final prototype is intentionally narrower than the complete concept. Working engineering evidence is prioritized over the number of sensors or features.

4. System Architecture and IoT Integration

The system is divided into three processing domains so that safety-critical control remains isolated from heavier perception tasks.

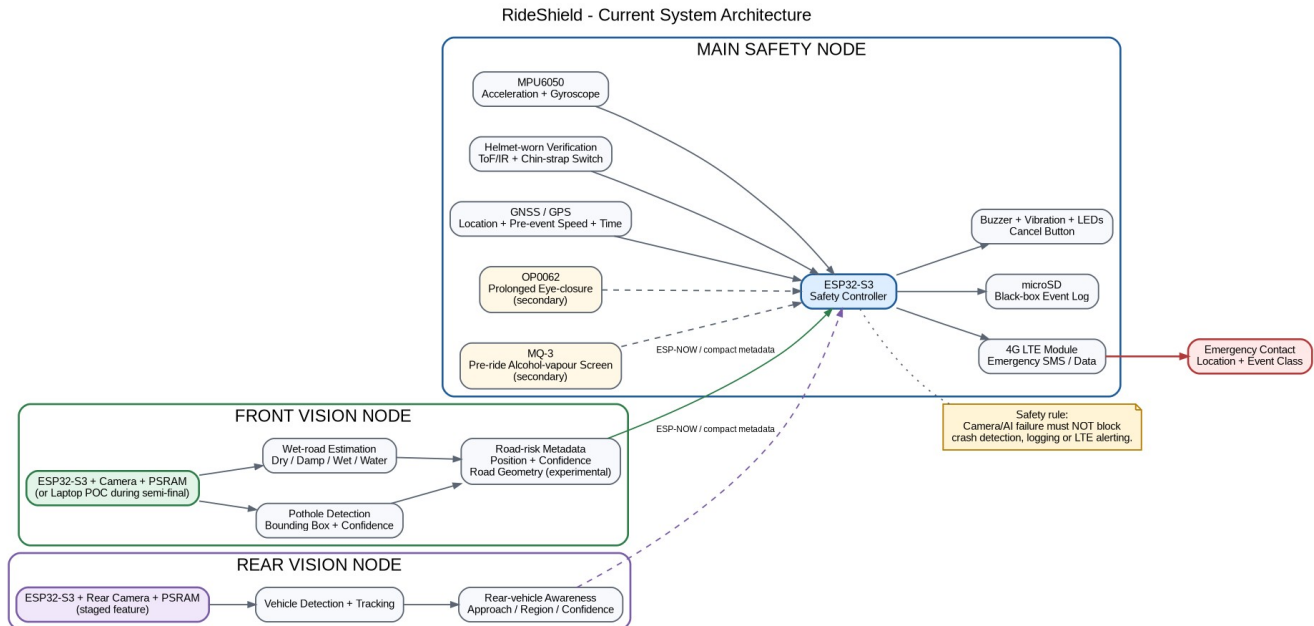


Figure 1. Current RideShield architecture for the Track A semi-final.

4.1 Main Safety Node

The main ESP32-S3 receives motion, helmet-worn and location inputs; runs the crash state machine; controls warnings and the rider-cancel flow; writes black-box data; and manages emergency communication. The core is designed to continue operating when vision nodes are unavailable.

4.2 Front Vision Node

A front-facing camera supplies road images to two perception functions: pothole detection and wet-road estimation. During the semi-final, the model may be demonstrated on a laptop if edge deployment is still under optimization. The final architecture targets an ESP32-S3 camera node with PSRAM and compact metadata exchange rather than raw-video streaming.

4.3 Rear Vision Node

The rear camera is a staged auxiliary subsystem for vehicle detection, tracking and approach awareness. Because the camera is helmet-mounted, head rotation changes the monitored region; this limitation will be handled through confidence reduction and further validation rather than claiming radar-like absolute blind-spot geometry.

5. Core Embedded Design

5.1 Crash-event detection and classification

The MPU6050 is used at its high acceleration range for prototype crash-event sensing. A single threshold is not sufficient. The controller evaluates acceleration/rotation anomalies, pre-event GPS speed where available, orientation change, post-event stillness and rider response. Because the MPU6050 can saturate in strong impacts, RideShield does not claim true impact force or medical injury severity.

Output terminology: Low-energy crash event / Moderate-energy crash event / High-energy crash event, with an internal Crash Event Severity Index (0-100). This is an event classification, not a medical diagnosis.

5.2 Crash state machine

State	Purpose
NORMAL	Monitor sensors and maintain the rolling black-box buffer.
IMPACT CANDIDATE	A motion anomaly is detected; collect a short validation window.
VERIFY	Check rotation/orientation, speed context and post-event movement.
COUNTDOWN	Activate warning and allow the rider to cancel a false alarm.
CONFIRMED EVENT	Calculate event class, freeze logs and prepare the alert.
ALERT / RETRY	Send location/event information; retain data locally if the network is unavailable.

5.3 Helmet-worn verification

The design uses two independent inputs: an internal proximity/distance sensor (ToF or suitable IR method) and a chin-strap switch. The helmet is reported as properly worn only when head presence and strap closure agree. This reduces false positives compared with using the strap alone.

5.4 Black-box logging

A rolling data buffer stores pre-event and post-event motion, GPS time/location/speed, helmet state, rider response and system status. On a confirmed event, the relevant window is written to microSD. Logging is non-blocking so storage activity does not stop crash detection.

5.5 Emergency communication

The LTE path sends an emergency message containing the event class and the best available location. If a fresh GPS fix is not available, the system can use the last valid location with its timestamp. LTE power transients are treated as a separate electrical-design risk so communication cannot brown out the main controller.

6. Computer-Vision Subsystems

6.1 Pothole detection

The front camera provides a road-region image to an object detector. Public pothole datasets are used for initial training, followed by local Sri Lankan road images from the intended camera viewpoint. The output is a bounding box, confidence and approximate position relative to the rider path. Model confidence alone is not interpreted as danger; position and speed are considered by the road-risk logic.

6.2 Wet-road estimation

A separate lightweight classifier estimates visible road-surface state such as dry, damp, wet or standing water. The system does not claim direct measurement of tyre-road friction. Wetness is treated as a risk indicator that can be combined with speed, pothole position and perception confidence.

6.3 Road-risk concept

The longer-term road-risk engine combines pothole hazard, wetness, speed and perception confidence. Experimental road geometry/grade estimation may be added later, but helmet motion makes camera-only grade estimation unreliable and therefore it is not a semi-final dependency.

7. Technology Stack and Major Components

Layer	Current choice / role
Main MCU	ESP32-S3 - safety controller and IoT integration
Motion	MPU6050 - acceleration and gyroscope
Helmet-worn	ToF/IR proximity + chin-strap switch
Position	GNSS/GPS module
Communication	4G LTE module; SMS/data
Storage	microSD black-box log
Front vision	Camera + Python/OpenCV/ML POC; ESP32-S3 edge node is the target
Rear vision	Rear camera + detection/tracking as a staged feature
Rider monitoring	OP0062 and MQ-3 are secondary features
Firmware	C/C++, ESP32 framework/FreeRTOS-style task separation
Vision software	Python, OpenCV, lightweight object detection and image classification
Node link	ESP-NOW or equivalent compact metadata transport

8. Current Semi-Final Progress

Status	Evidence / current position
Validated design	System scope revised and architecture separated into safety and vision nodes.
Software POC	Pothole computer-vision workspace contains model weights and a demo/inference pipeline; quantitative test metrics are still to be documented.
Dataset / model plan	Pothole and wet-road dataset strategy established; local Sri Lankan data collection is planned for fine-tuning.
Hardware design	Main component set and interfaces selected: ESP32-S3, MPU6050, helmet-worn sensors, GPS, LTE, microSD, warnings.
In integration	Crash state machine, GPS/LTE, black-box logging and helmet-worn logic require component-level/live validation before the final.
Staged	Rear vehicle awareness, OP0062 and MQ-3 are lower-priority integrations after the safety core/front-vision POC.
Evidence rule: Only features demonstrated or measured should be described as working. Incomplete subsystems are intentionally labelled "in integration" or "staged".	

9. Semi-Final Validation Plan

Test	Changed input / condition	Expected evidence
Helmet-worn	Head presence and strap state changed independently	Only both-valid condition reports properly worn
Crash rejection	Normal movement / tilt / handling	No emergency event
Crash candidate	Controlled impact on safe fixture/dummy	State changes to candidate/verify/countdown
Cancel flow	Press cancel during countdown	Alert cancelled and event logged
Confirmed event	Do not cancel	Event class + local log + emergency alert attempt
Pothole POC	Normal-road and pothole images/video	Bounding box/confidence only on detected potholes
Wet-road POC	Dry vs visibly wet road samples	Wetness class/confidence

Measured results should be added to the GitHub /results folder as soon as they are available. Even imperfect results are reported honestly because the semi-final is a validation stage rather than a production-readiness assessment.

10. Current Limitations and Engineering Risks

Risk / limitation	Engineering response
MPU6050 ± 16 g limit	Use event classification and saturation/duration/context, not claimed true impact force.
Helmet-mounted camera motion	Treat road/rear geometry as confidence-limited; validate with local footage.
Wet road is not friction	Report wet-road risk, not measured slipperiness coefficient.
LTE current spikes	Use a dedicated high-current supply path and local capacitance; test for brownouts.
GPS availability	Use current or last valid fix with timestamp; never block local logging.
AI generalization	Fine-tune and test on Sri Lankan roads, rain, lighting and helmet-camera viewpoints.
Camera lens contamination	Future enclosure/lens protection and image-quality checks.
Feature scope	Safety core and front vision are prioritized; secondary features remain staged.

11. Planned Improvements Before the Final

- Complete and repeatedly validate the end-to-end crash -> countdown -> location -> alert -> black-box path.
- Record quantitative false-alarm, missed-event, SMS-latency and brownout/reset results.
- Fine-tune pothole and wet-road models using locally captured road footage and create an unseen test split.
- Optimize/quantize front-vision inference for the ESP32-S3 edge node if performance is sufficient.
- Integrate and validate the rear-camera awareness subsystem after the core path is stable.
- Add rider-monitoring features and improve power, enclosure, weather protection and mechanical integration.

12. Final Engineering Position

RideShield is being developed as a fault-isolated embedded IoT safety platform rather than a collection of independent sensors. The semi-final prototype focuses on proving the safety-critical flow and a credible road-perception proof-of-concept. The final stage will emphasize integration reliability, quantitative validation and edge deployment of the perception functions.

Track A focus: Hardware integration + correct firmware behavior + changed-input physical testing. This proposal intentionally aligns the development plan with those three validation areas.